Nombre A: Daniel Bernal Anzano **DNI A**: 73514866N

Nombre B: David Araiz Saez **DNI B**: 26587036W

Asignatura: DAWM2D (OPIA) **Fecha**: 13/11/2026

Instrucciones:

Se valorará el orden y limpieza. Una respuesta incomprensible se considerará no respondida. Es obligatorio respetar el formato de entrega. **Todos** los recursos **deben** ser entregados. El incumplimiento supondrá una calificación de cero.

Procura ser breve y responder a todo lo que se pide.
Explicar (brevemente) significa 2-3 líneas. Indicar (brevemente) significa 1-2 líneas. Nombrar significa 1 palabra. La respuesta **solo debe** añadirse después de cada pregunta; en otro lugar supondrá una calificación de cero.

Cualquier tipo de fraude o copia supondrá una calificación de cero.
Lee antes de empezar y analiza los recursos proporcionados. Organiza el tiempo.

Formato de entrega:

Una única carpeta, que será comprimida y entregada en un archivo .zip.
Este documento resuelto por el alumno, en la raíz de dicha carpeta, en .pdf y .docx.

Si es necesario, dentro de la carpeta principal habrá una o varias subcarpetas que incluirán **TODOS los recursos adicionales pedidos, desarrollados y usados**. Los recursos no entregados supondrán una calificación de cero en el ejercicio donde se pidan.

Las capturas deben incluir el nombre del autor y la palabra P3 (ej: Juan Perez P3).
Los recursos se deben organizar en subcarpetas por ejercicios.

Formato mínimo:
dawm2d_opia_p3.zip
|.....dawm2d_opia_p3.pdf (tarea, en pdf)
|.....dawm2d_opia_p3.docx (tarea, en docx)
|.....recursos/ (carpeta para recursos adicionales, pedidos, desarrollados y usados)
|.....ejercicio_n/ (subcarpeta para recursos del ejercicio n)

Instrucciones.
La práctica se realizará en parejas y deberá incluir el nombre de ambos integrantes.

Evaluación.
Todos los apartados (CE) se ponderan por igual en la nota del RA.

PRÁCTICA 3 - Conexión entre la aplicación y el servicio de IA [RA3] v0

Usa la plantilla de código adjunta para completar esta práctica.

- **app.py:** server FastAPI básico con predicción. Igual al de práctica 2.
- **streamlit_app.py:** web Streamlit con una interfaz básica. Llama a app.py.

Puedes incluir una sola versión de **streamlit_app.py** con todos los cambios acumulados en el ejercicio final.

a) Conexión app-servicio de IA

Configura la conexión entre la aplicación Streamlit y tu servicio de IA: indica la URL pública de tu API en Render y envía los datos en formato JSON mediante una petición POST al endpoint `/predict`. Despliega app.py y streamlit_app.py.

Entregable (memoria): una captura de las aplicación Streamlit (streamlit_app.py) y FastAPI (app.py) en Render. El log de las app. Las url del servicio FastAPI y Streamlit en Render.

Respuesta apartado a:

1. Conexión con la API:

La URL de la API se lee de la variable de entorno API_URL y también se puede cambiar en la barra lateral. Los datos se envían en JSON con un POST a /predict:

```
response = requests.post(api_url + "/predict", json=payload,
timeout=TIMEOUT)
```

2. Despliegue en Render:

Se han creado dos Web Services de Python (plan Free, Frankfurt) desde el repositorio danielbernalanz/practica3california, rama main, dentro del proyecto "practica 3 california". Como el código está en recursos/ejercicio_e/codigo, los comandos entran primero en esa carpeta.

• API: build "pip install -r recursos/ejercicio_e/codigo/requirements.txt" y start "cd recursos/ejercicio_e/codigo && uvicorn app:app --host 0.0.0.0 --port $PORT".

• Streamlit: mismo build y start "cd recursos/ejercicio_e/codigo && streamlit run streamlit_app.py --server.port $PORT --server.address 0.0.0.0 --server.headless true". Lleva la variable API_URL con la URL de la API.

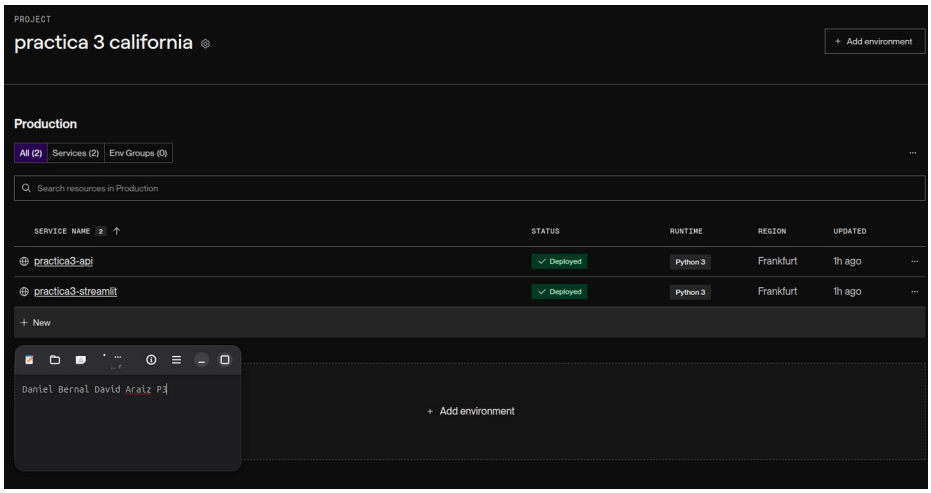
3. URLs:

• FastAPI (app.py): https://practica3-api-5qnf.onrender.com (documentación en /docs)

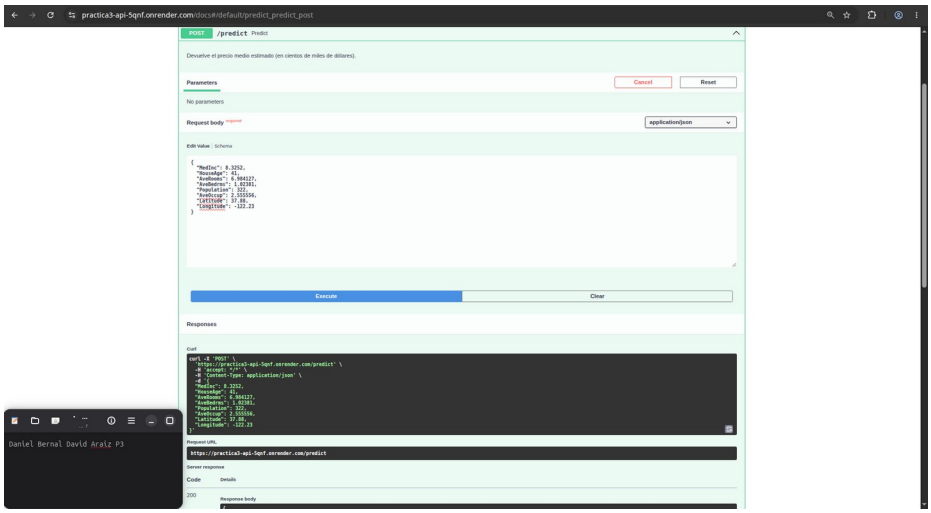
• Streamlit (streamlit_app.py): https://practica3-streamlit.onrender.com

4. Capturas:

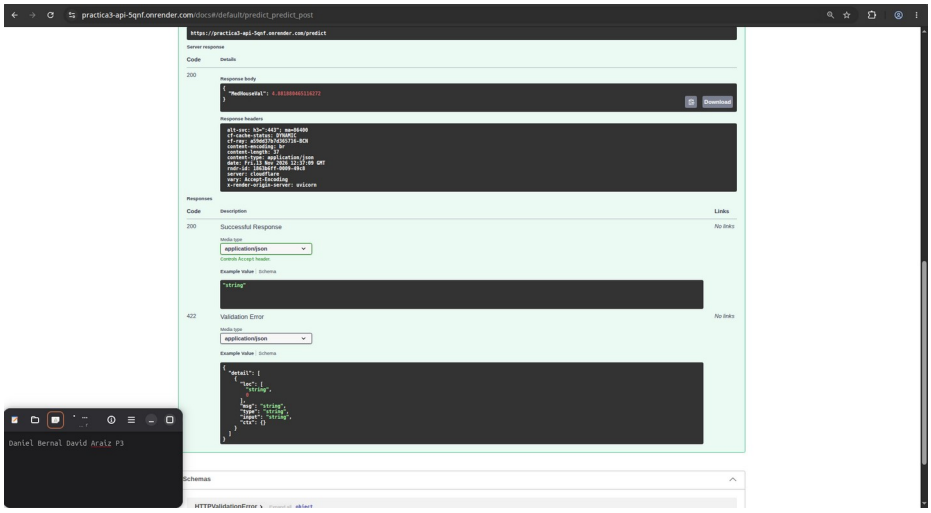
• *Los dos servicios en Render, con estado Deployed:*



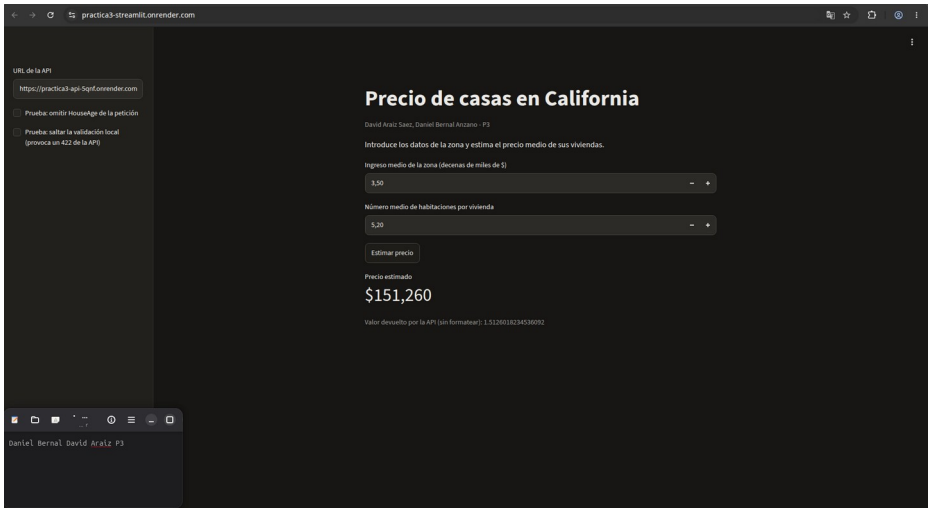
• *FastAPI en Render, probando POST /predict desde /docs con una petición real:*



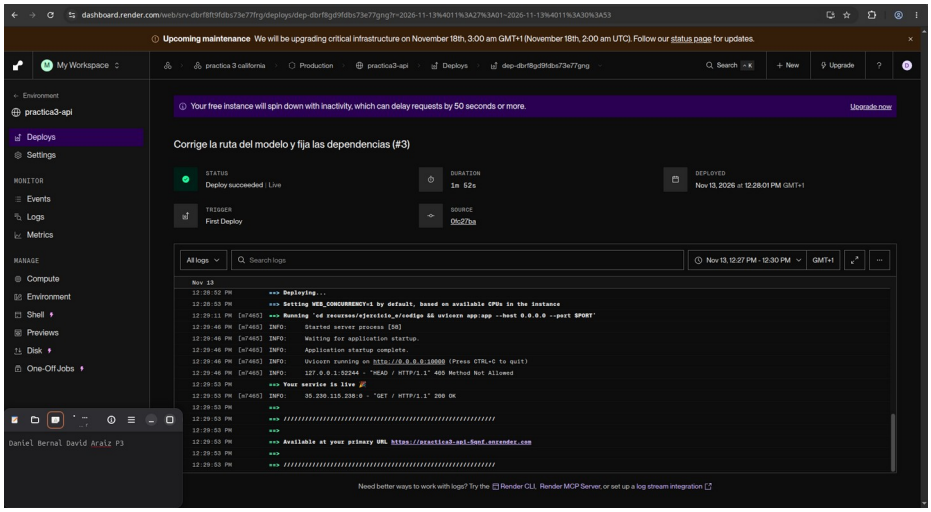
• *Respuesta de la API (200 con MedHouseVal):*



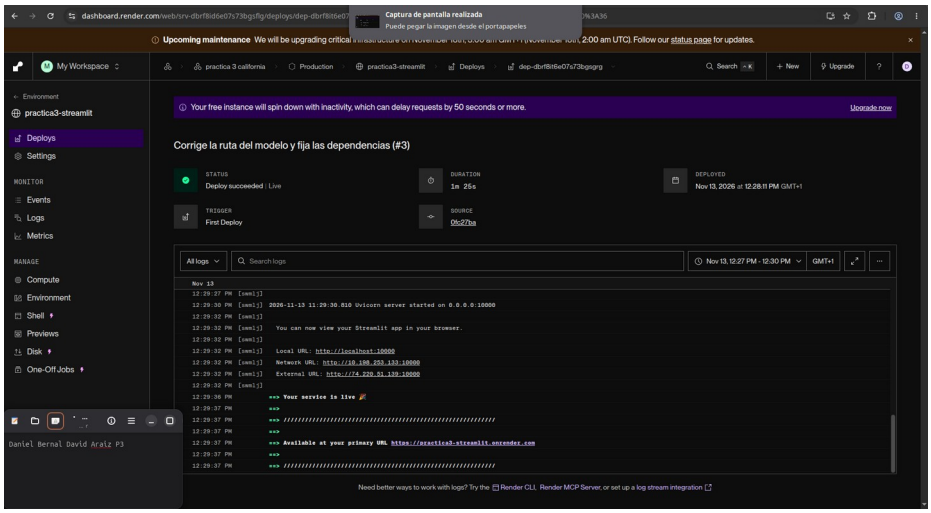
• *Streamlit en Render con un caso real:*



• *Log de la API:*



• *Log de Streamlit:*



b) Preparación de los datos de entrada

La aplicación solo pide al usuario algunas variables, así que define valores por defecto razonables para el resto. Antes de enviar la petición, valida que contiene las 8 variables que necesita el modelo (MedInc, HouseAge, AveRooms, AveBedrms, Population, AveOccup, Latitude, Longitude); si falta alguna, la aplicación no envía la petición y muestra qué falta. Completa el código faltante.

Entregable (ZIP, memoria): el código. Dos pruebas: una petición válida y otra con un campo faltante, mostrando el resultado de cada una.

Respuesta apartado b:

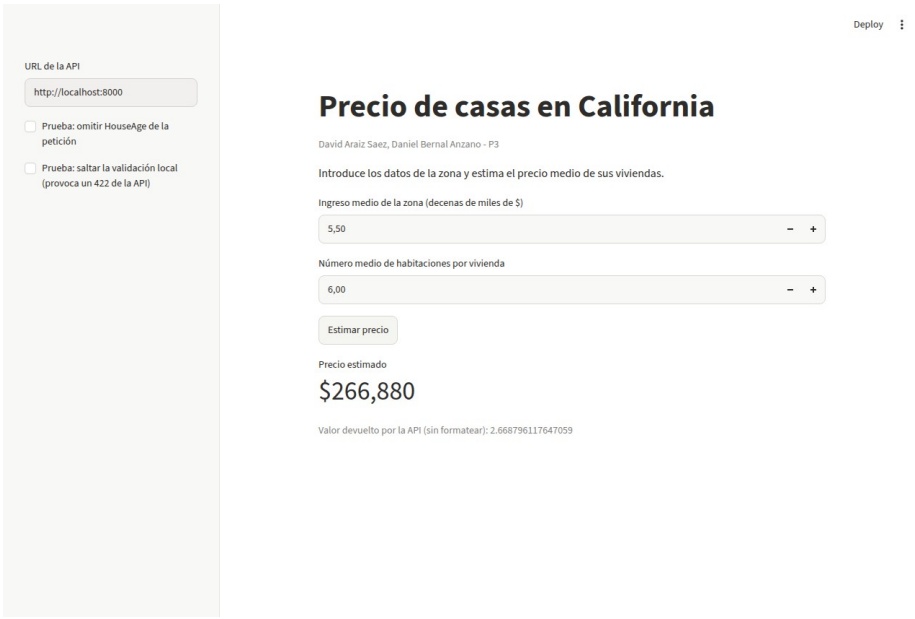
1. Lógica de validación previa al envío:

La función find_missing(payload) comprueba que todas las variables requeridas por el modelo (FEATURE_NAMES) existan en el diccionario y que su valor no sea None (payload.get(nombre) is None). Se garantiza que 0.0 sea aceptado como dato válido. Si falta alguna variable obligatoria, la petición no se envía a la API (st.stop()) y se notifica al usuario qué campos faltan.

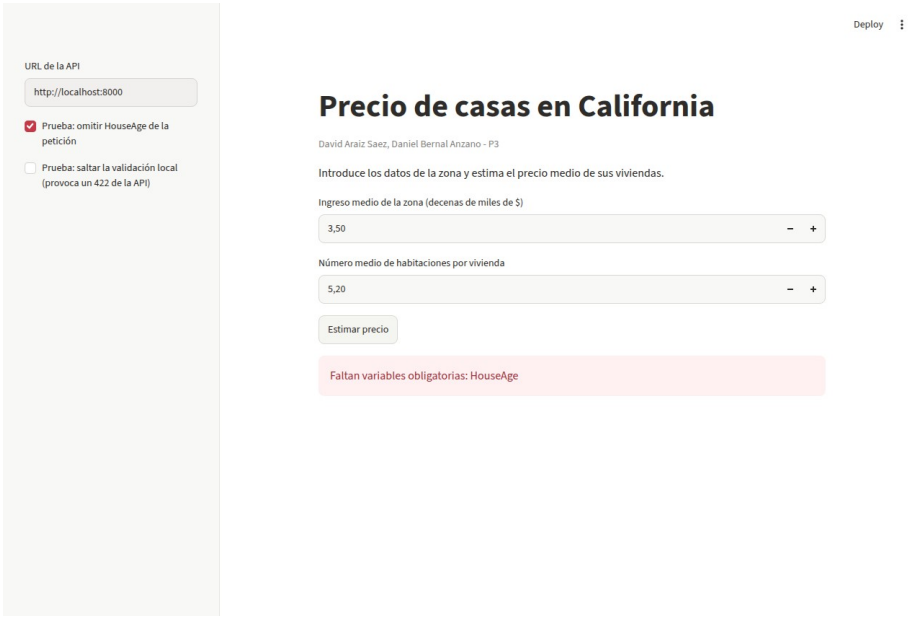
```
def find_missing(payload: dict) -> list[str]:
    return [nombre for nombre in FEATURE_NAMES if payload.get(nombre) is None]
```

2. Pruebas realizadas:

• *Prueba con petición válida: se envían los 8 parámetros con datos consistentes. La API responde HTTP 200 con la predicción:*



• *Prueba con campo faltante: al omitir HouseAge desde la barra lateral, la validación local cancela el envío y muestra el mensaje indicando la variable ausente:*



c) Transformación de los datos de salida

Transforma la respuesta de la API a un formato legible (precio en dólares) antes de mostrarla. Recuerda que la variable objetivo del dataset está en cientos de miles de dólares. Completa el código faltante.

Entregable (ZIP, memoria): el código. Un ejemplo que muestre el valor devuelto por la API y el precio ya formateado que ve el usuario.

Respuesta apartado c:

1. Transformación a formato legible en dólares:

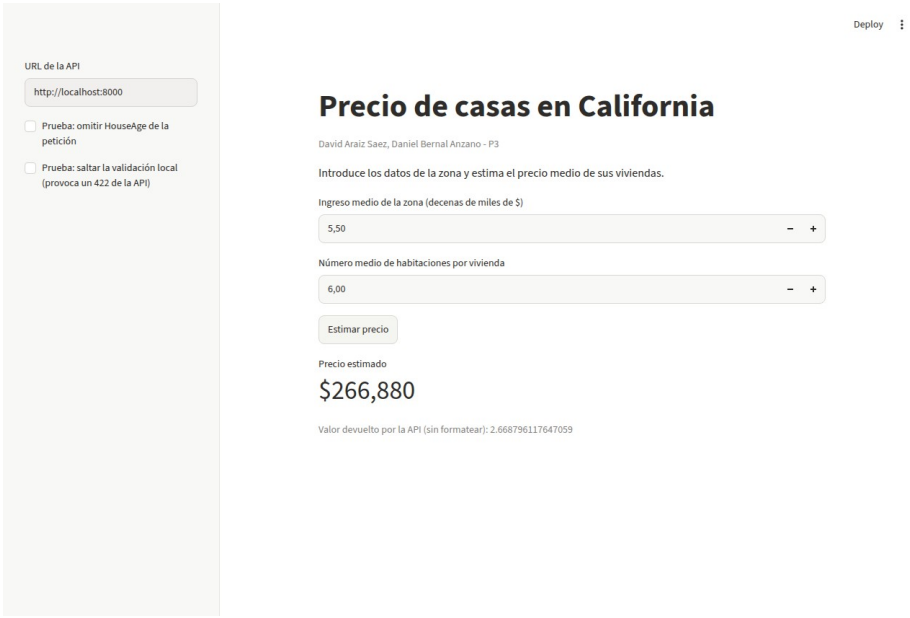
Dado que la variable objetivo MedHouseVal está expresada en cientos de miles de dólares, se multiplica el valor devuelto por 100.000 y se aplica formato con separador de miles mediante un f-string.

```
def format_price(med_house_val: float) -> str:
    precio = med_house_val * 100_000
    return f"${precio:,.0f}"
```

2. Ejemplo de ejecución (MedInc = 5.5 y AveRooms = 6.0):

• Valor crudo devuelto por la API FastAPI: 2.668796117647059
• Transformación: 2.668796117647059 × 100.000 = 266.879,61 $
• Precio formateado presentado en interfaz: $266,880

En la interfaz el precio sale en grande y, debajo y más pequeño, aparece el valor que devuelve la API:



d) Diseño de interfaz de usuario

Completa la interfaz de Streamlit para que permita introducir las variables clave (ingreso medio, número de habitaciones).

Entregable (ZIP, memoria): el código. Una captura de la interfaz funcionando con un caso de prueba real.

Respuesta apartado d:

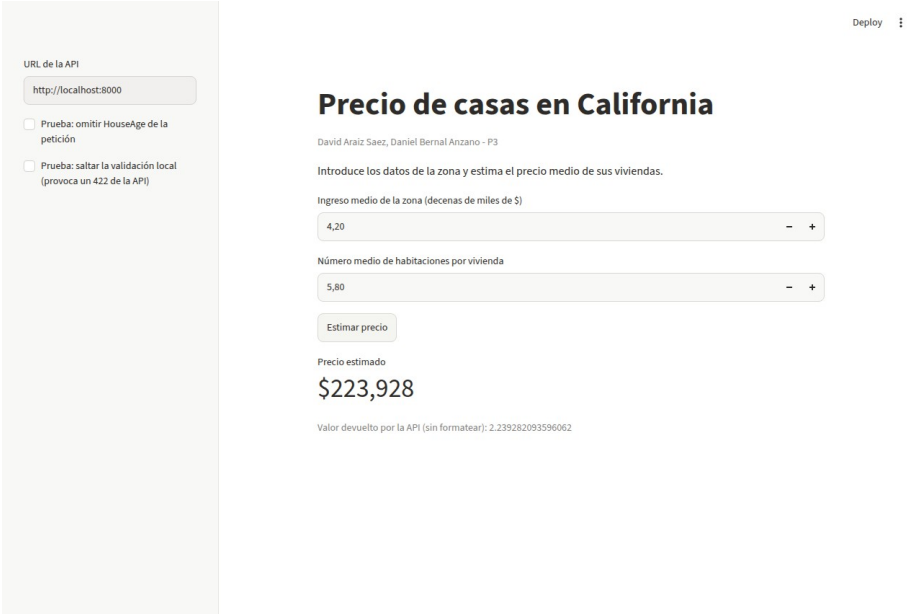
1. Integración de control para número de habitaciones:

Se reemplazó el valor fijo por un componente interactivo st.number_input que permite ingresar el número medio de habitaciones por vivienda (AveRooms), parametrizado con min_value=1.0, max_value=15.0, value=5.2 y step=0.1.

```
ave_rooms = st.number_input(
    "Número medio de habitaciones por vivienda",
    min_value=1.0, max_value=15.0, value=5.2, step=0.1,
)
```

2. Caso de prueba real:

Se ingresan los valores MedInc = 4.2 y AveRooms = 5.8. Al pulsar 'Estimar precio', la API devuelve la predicción y sale un precio estimado de $223,928:



e) Documentación y control de errores

Añade control de errores a la aplicación (respuestas de error de la API) con mensajes claros para el usuario.
Documenta la API en un README con un ejemplo de petición y de respuesta.

Entregable (ZIP, memoria): el código completo usado (con todos los cambios completos de `streamlit_app.py` de todos los ejercicios) en el ZIP. El README con la documentación de la API incluida (ZIP). Una prueba que provoque un error controlado, por ejemplo cambiando la URL de la API (memoria).

Respuesta apartado e:

1. Control de errores:

La llamada a la API va dentro de un try/except. Si no hay respuesta (sin conexión, servicio caído o más de 90 segundos de espera) se muestra un mensaje diciendo que no se pudo contactar con la API. Si la API responde con un código distinto de 200 (422, 404, 500...) se muestra el código y el texto que devuelve. Se usa response.text y no response.json() porque en un error 500 el cuerpo no es JSON.

```
st.error(f"Error {response.status_code} devuelto por la API:
{response.text}")
```

2. Documentación de la API:

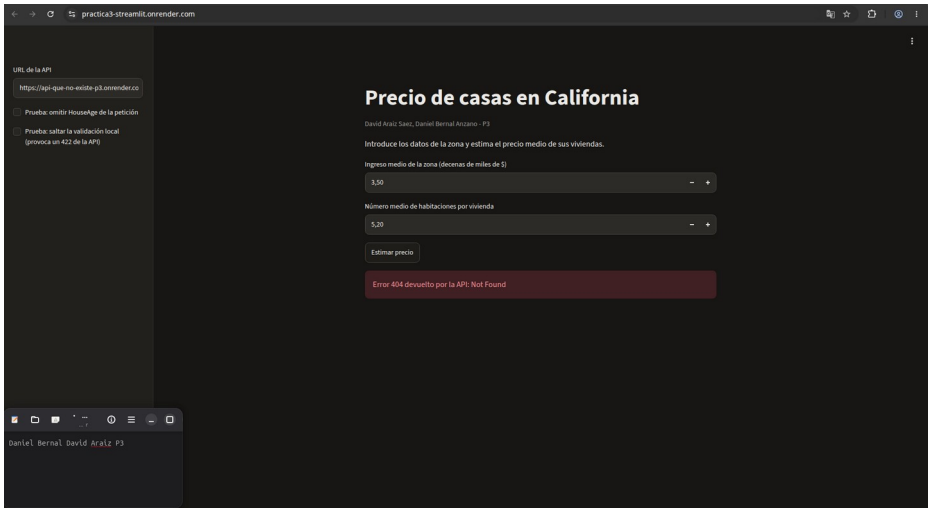
En el README del código (recursos/ejercicio_e/codigo/README.md, incluido en el ZIP) está explicado qué hace la aplicación, las URLs de Render, cómo ejecutarla en local, los comandos de despliegue y el endpoint POST /predict con un ejemplo de petición y de respuesta, además de una tabla con los errores.

• *Petición: {"MedInc": 8.3252, "HouseAge": 41, "AveRooms": 6.984127, "AveBedrms": 1.02381, "Population": 322, "AveOccup": 2.555556, "Latitude": 37.88, "Longitude": -122.23}*

• *Respuesta (200): {"MedHouseVal": 4.881880465116272}*

3. Prueba de error controlado:

• *Se cambia la URL de la API por https://api-que-no-existe-p3.onrender.com. Render contesta con un 404 porque ese servicio no existe y la aplicación no se rompe, muestra "Error 404 devuelto por la API: Not Found":*



• *Se marcan las casillas de omitir HouseAge y saltar la validación local, así la petición sale sin HouseAge y la API responde con un 422. La aplicación enseña el código y el detalle del error:*

